

Objets Personnalisés ([PSCustomObject]) et Tableaux de Bord HTML

1. Objectif

Dans l'administration quotidienne, les informations utiles pour évaluer la santé d'une machine ou d'un parc ne proviennent jamais d'une commande unique :

- L'espace disque provient du système de fichiers ou des lecteurs (Get-PSDrive),
- La mémoire et les processus proviennent de Get-Process,
- La disponibilité des services provient de Get-Service,
- L'identité et le système proviennent des variables d'environnement ou de l'instrumentation système.

Si vous vous contentez d'afficher ces résultats avec Write-Host ou de stocker chaque information dans des variables scalaires isolées, **vos données sont perdues pour l'automatisation** : impossible de les filtrer, de les trier, de les exporter proprement en CSV, ou de les intégrer dans un pipeline d'exploitation. De même, les tables de hachage classiques (@{}) désordonnent les clés et corrompent les exports tabulaires.

La compétence clé de ce TP consiste à abandonner ces pratiques non-structurées pour agréger ces informations disparates au sein d'une structure de données standardisée : **l'objet personnalisé ([PSCustomObject])**. Vous apprendrez ensuite à convertir ces objets en un **tableau de bord web HTML/CSS moderne et dynamique**.

2. Phase 1 : La Donnée Non-Structurée vs [PSCustomObject]

Vous allez expérimenter concrètement les deux erreurs d'architecture les plus fréquentes afin de mesurer l'apport de [PSCustomObject].

2.1 Expérimentation de l'impasse Write-Host

1. Rédigez une fonction de test rapide nommée Get-InfoDisqueTest qui calcule l'espace libre sur C: et tente de l'afficher via :
Write-Host "Disque C: \$EspaceLibre Go"
2. Dans la console, tentez de capturer le résultat de votre fonction dans une variable.

2.2 Le piège de la Table de Hachage face à Export-Csv

1. Déclarez une collection contenant deux tables de hachage classiques (@{}) décrivant deux serveurs :

```
$MonParcHashtable = @(
    @{ Serveur = "SRV-AD01"; RAM_Go = 32; Statut = "OPTIMAL" },
    @{ Serveur = "SRV-FILE01"; RAM_Go = 16; Statut = "ALERTE" }
)
```
2. Exportez directement cette collection dans un fichier CSV de test :

```
$MonParcHashtable | Export-Csv -Path "C:\Audit\test_hash.csv" -NoTypeInfo
```
3. Ouvrez le fichier généré avec le Bloc-notes (notepad C:\Audit\test_hash.csv).
4. **Questions d'analyse :**
 - Retrouvez-vous les colonnes Serveur, RAM_Go et Statut ?
 - Quelles colonnes apparaissent à la place (ex: IsReadOnly, Keys, Count) ?
 - Pourquoi Export-Csv produit-il ce résultat sur une table de hachage ?

2.3 Résolution immédiate avec [PSCustomObject]

1. Transformez la collection précédente en utilisant le transtypeur d'objet personnalisé [PSCustomObject]@{ ... }.
2. Affichez votre collection à l'écran : constatez l'ordre strict des colonnes et la disposition tabulaire native.
3. Exportez à nouveau cette collection vers un fichier CSV (test_custom.csv).
4. Vérifiez le contenu du fichier : constatez la présence exacte des en-têtes et des enregistrements métiers.
5. Validez le type .NET réel de l'objet à l'aide de sa méthode .GetType().FullName.

3. Phase 2 : Consolidation Multi-Sources et Métriques Calculées

Vous allez maintenant concevoir une sonde d'audit complète pour votre poste local en agrégeant trois sous-systèmes distincts au sein d'un enregistrement unique.

3.1 Collecte et normalisation des grandeurs

Rassemblez au sein d'un script de test les métriques suivantes :

1. **Identité et Temps :**
 - Nom de la machine locale (\$env:COMPUTERNAME).
 - Horodatage du contrôle sous la forme standardisée "yyyy-MM-dd HH:mm".
2. **Sonde de Stockage (Lecteur C:) :**
 - Interrogez le lecteur système à l'aide de Get-PSDrive -Name C.

- Calculez la taille totale du disque et la quantité d'espace disponible converties en gigaoctets, arrondies à une décimale via `[math]::Round()`.
 - Calculez le ratio de remplissage ou le pourcentage d'espace libre restant.
3. **Sonde de Service Critique :**
 - Interrogez l'état d'un service système essentiel (ex : Spooler ou wuauserv).
 - Isolez son état opérationnel (Running ou Stopped).
 4. **Sonde Processus :**
 - Identifiez le processus le plus consommateur de mémoire vive sur la machine à l'aide de `Get-Process`.
 - Extrayez son nom et sa mémoire de travail (`WorkingSet64`) convertie en mégaoctets.

3.2 Construction du Payload Métier et Arbitrage de Santé

1. Fusionnez ces grandeurs hétérogènes dans un unique `[PSCustomObject]` nommé `$Diagnostic`.
2. **Indicateur de Santé Dynamique :**
 - Ajoutez une propriété calculée nommée `EtatGlobal`.
 - Règle d'arbitrage logique :
 - Si l'espace libre sur C: est strictement inférieur à 15% **OU** si le service critique est arrêté : la valeur doit être "CRITIQUE".
 - Sinon : la valeur doit être "OPTIMAL".
3. Affichez votre objet personnalisé à l'écran sous forme de tableau propre avec `Format-Table -AutoSize`.
4. Démontrez que cet objet est immédiatement filtrable dans le pipeline en exécutant : `$Diagnostic | Where-Object EtatGlobal -eq "OPTIMAL"`

4. Phase 3 : Restitution Visuelle Web (ConvertTo-Html)

Un compte-rendu en console ne convient pas pour un tableau de bord partagé avec l'exploitation ou la direction. Vous allez convertir vos données en une page web HTML autonome et stylisée.

4.1 La cmdlet ConvertTo-Html et ses limites par défaut

1. Envoyez votre objet de diagnostic vers la commande `ConvertTo-Html` et enregistrez le flux dans `C:\Audit\brut.html`.
2. Ouvrez ce fichier dans votre navigateur web (`Invoke-Item C:\Audit\brut.html`).
3. **Constat :**
 - Quelles sont les lacunes majeures du rendu visuel par défaut (police, bordures, espacements) ?
 - Pourquoi un document HTML brut produit par PowerShell est-il inadapté à une présentation professionnelle ?

4.2 Injection d'une feuille de style CSS embarquée

Pour rendre le document autonome (consultable sans connexion Internet et transférable par email sans perte de mise en forme), la feuille de style CSS doit être **embarquée directement dans l'en-tête HTML**.

1. Déclarez une chaîne de caractères multilignes contenant une feuille de style CSS moderne :
 - Une typographie sans empattement propre (*Segoe UI, Arial, sans-serif*).
 - Un tableau bordé, centré, avec des marges intérieures (*padding*) soignées.
 - Un en-tête de tableau sombre et contrasté.
 - Une alternance de couleur sur les rangées paires et impaires (*zebra-striping : #f7fafc*).
 - Deux classes de badges d'état distinctes : *.badge-optimal* (vert doux) et *.badge-critique* (rouge doux).
2. Injectez cette feuille de style dans le paramètre `-Head` de `ConvertTo-Html`.
3. Générez à nouveau le fichier et observez l'amélioration visuelle dans votre navigateur.

4.3 Architecture modulaire par fragments (-Fragment)

En production, un tableau de bord complet comporte plusieurs sections distinctes (un récapitulatif synthétique, un tableau d'état des disques, une table des processus, et un pied de page horodaté).

1. Exécutez `ConvertTo-Html` avec le paramètre `-Fragment` : que constatez-vous concernant les balises de structure `<html>`, `<head>` et `<body>` ?
2. Expérimentez l'assemblage modulaire d'un rapport complet en concaténant :
 - Une section de titre et d'horodatage (`<h1>`, `<p class='horodatage'>`).
 - Un fragment tabulaire pour le diagnostic général.
 - L'encapsulation finale dans un document HTML complet via un appel unique à `ConvertTo-Html -Body ... -Head ...`

4.4 Coloration dynamique des alertes (Post-traitement)

Par conception, `ConvertTo-Html` génère des cellules textuelles brutes `<td>CRITIQUE</td>` sans y associer de couleur ou de classe CSS.

1. Utilisez l'opérateur de substitution textuelle **-replace** sur le code HTML généré pour encapsuler les mentions textuelles dans des conteneurs dotés de classes CSS :
 - Remplacer `<td>OPTIMAL</td>` par `<td><span class="badge-optimal">OPTIMAL</span></td>`.
 - Remplacer `<td>CRITIQUE</td>` par `<td><span class="badge-critique">CRITIQUE</span></td>`.
2. Générez le rapport avec cette substitution et vérifiez que les alertes ressortent désormais immédiatement à l'écran.

5. Phase 4 : Industrialisation du Script

(generer_dashboard.ps1)

Vous allez concevoir le script d'administration final nommé **generer_dashboard.ps1**.

Ce script réalisera l'audit d'une liste de volumes ou de cibles, synthétisera l'état de chaque cible au sein d'une collection de [PSCustomObject], et produira un tableau de bord HTML autonome avec coloration conditionnelle des statuts.

5.1 Cahier des charges technique et fonctionnel

1. En-tête et Paramètres :

- Présence obligatoire de l'attribut [CmdletBinding()].
- [string[]]\$Lecteurs = @("C", "D") : liste des lecteurs à inspecter (valeur par défaut : au moins deux lettres de disques).
- [int]\$SeuilAlerteDisquePct = 15 : seuil minimum d'espace disponible en pourcentage déclenchant l'alerte.
- [string]\$CheminRapport = "C:\Audit\dashboard_sante.html" : emplacement du fichier HTML de sortie.
- [switch]\$OuvrirNavigateur : commutateur déclenchant l'ouverture automatique du rapport à la fin du traitement.

2. Collecte et Résilience :

- Le script parcourt l'ensemble des lecteurs demandés.
- Pour chaque lecteur, il interroge les ressources via Get-PSDrive ou CIM :
 - Si le lecteur existe : capture de l'espace total, de l'espace libre et du pourcentage disponible.
 - Si le lecteur est introuvable ou non monté : l'anomalie est gérée proprement sans planter le script (valeurs à "N/A" et statut "INTROUVABLE").
- Il relève également le processus système le plus gourmand en mémoire à l'instant du contrôle.

3. Construction des Objets Métiers :

- Chaque volume audité produit un [PSCustomObject] comportant :
 - Lecteur : identifiant du volume inspecté.
 - DateControle : heure exacte du relevé (yyyy-MM-dd HH:mm).
 - TailleTotaleGo : capacité totale formatée.
 - EspaceLibreGo : espace libre restant.
 - RatioLibre : pourcentage d'espace disponible formaté avec le symbole %.
 - TopProcessus : nom du processus le plus lourd et sa consommation en Mo.
 - Statut : "OPTIMAL" si le pourcentage libre dépasse le seuil, "CRITIQUE" s'il franchit le seuil à la baisse, ou "INTROUVABLE" si le lecteur n'existe pas.

4. Génération Web et Coloration :

- Les styles CSS doivent être entièrement intégrés dans la page.
- Post-substitution des statuts pour afficher les badges colorés correspondants.
- Surlignage discret de la rangée (<tr>) complète en cas d'alerte critique.

5. Gestion de l'Environnement et Clôture :

- S'assurer que le dossier de destination (ex : C:\Audit) est automatiquement créé s'il est absent.
- Sauvegarder le fichier HTML avec l'encodage **UTF-8** pour garantir l'affichage correct des caractères accentués (*Mémoire, Arrêté, Généré*).
- Si le commutateur -OuvrirNavigateur est actif, lancer la page avec Invoke-Item.

5.2 Exemple d'Exécution Attendue

1. Génération standard du tableau de bord

```
.\generer_dashboard.ps1 -Lecteurs "C", "D" -SeuilAlerteDisquePct 20
```

2. Génération personnalisée avec ouverture automatique du navigateur

```
.\generer_dashboard.ps1 -CheminRapport "C:\Audit\supervision.html" -OuvrirNavigateur
```

Sortie console attendue :

=== GÉNÉRATION DU TABLEAU DE BORD D'INFRASTRUCTURE ===

Lecteurs analysés : C, D

Seuil d'alerte configuré : 20 %

Collecte des métriques en cours...

Lecteur	DateControle	TailleTotaleGo	EspaceLibreGo	RatioLibre	TopProcessus	Statut
C	2026-09-09 11:30	120.0 Go	45.2 Go	37.7 %	sqlservr (512.4 Mo)	OPTIMAL
D	2026-09-09 11:30	500.0 Go	42.1 Go	8.4 %	backup_agent (85 Mo)	CRITIQUE

Tableau de bord HTML généré : C:\Audit\dashboard_sante.html

Ouverture du tableau de bord dans le navigateur...